

Question 1: In your own words, what is the difference between a delegate and a lambda expression?

1. Definition & Core Difference:

- A **Delegate** is a *type definition / contract* in C# that defines the signature (parameters and return type) of a callable method. It acts as a type-safe function pointer or specification.
- A **Lambda Expression** is an *inline, anonymous method implementation* written using the arrow syntax ($\Rightarrow$). It provides the actual executable logic that conforms to a delegate's contract.

2. Analogy & Relationship:

The delegate defines *what shape* of method is expected, whereas the lambda expression is the *actual logic* passed into that shape without needing to declare a standalone named method.

3. Code Demonstration:

```
// 1. Delegate declaration (The Contract)
delegate bool FilterDelegate(int number);
// 2. Lambda expression assigned to the delegate (The Implementation)
FilterDelegate isEven = x => x % 2 == 0;
```

Key Takeaway: A delegate is the **Type**; the lambda expression is the **compact implementation**.

Question 2: Why is var still statically typed even though the type isn't written explicitly?

1. Compile-Time Type Inference:

In C#, var is **not** a dynamic type and does **not** defer type resolution to runtime. Instead, the compiler inspects the expression on the right-hand side of the assignment at **compile-time** and infers the exact, strongly-typed type.

2. Identical IL (Intermediate Language) Output:

When the C# compiler produces the binary assembly (.dll/.exe), `var age = 25;` and `int age = 25;` generate the **exact**

same IL code (System.Int32). There is zero runtime performance penalty.

3. Strict Type Safety (No Re-assignment to Other Types):

Once inferred, the variable's type is locked. Trying to assign a different type later in the same scope triggers a compilation error:

```
var count = 10; // Inferred as int
count = "Hello"; // COMPILER ERROR: Cannot implicitly convert type 'string' to 'int'
```

Contrast with dynamic: Unlike *dynamic*, which bypasses compile-time checks, *var* retains 100% full compile-time type

safety, IntelliSense support, and static analysis.

Question 3: Give one real scenario where an anonymous type is more convenient than creating a full class.

1. Practical Real-World Scenario: Data Shaping & Projections (LINQ & Reports)

The most common real-world use case is **data projection** when querying databases or APIs. Often, an entity contains numerous columns, but the current UI view, report, or dropdown only needs two or three fields.

2. Concrete Example:

Suppose a database table Customer has 20+ columns (Id, PasswordHash, Salt, CreditCardToken, BillingAddress, AuditTimestamps, etc.). For an autocomplete search dropdown, we only require the customer's **FullName** and

Email:

```
// Projecting into a lightweight anonymous type on the fly:
var customerCard = new {
    customer.FullName,
    customer.Email
};
```

3. Why It Is More Convenient:

- **Avoids Class Bloat:** Eliminates the need to create, name, and maintain dozens of single-use DTO classes (e.g., CustomerDropdownSummaryDTO).
- **Read-Only Safety:** Anonymous types are immutable (read-only), guaranteeing that the projected values cannot be accidentally modified in memory.
- **Cleaner Codebase:** Keeps temporary transformation logic localized exactly where it is used